



Powered by
Arizona State University®

LOGICA DE PROGRAMACION

TRABAJO FINAL

**CineReserva – Sistema de Gestión de Asientos para
Salas de Cine**

DOCENTE:

CABEZAS ERAZO DARIO SEBASTIAN

ESTUDIANTE:

LINCANGO MOROCHO MAYERLY ABIGAIL

Repositorio GitHub: [https://github.com/Abigailuchi/sistema-
asientos-cine](https://github.com/Abigailuchi/sistema-asientos-cine)

FECHA DE ENTREGA:

27-06-2026



1. INTRODUCCION

En la actualidad la tecnología está metida en casi todo lo que hacemos, aunque la mayoría de las veces ni nos damos cuenta. Usamos la tecnología en acciones cotidianas como comunicarnos, realizar pagos, revisar información en línea o comprar entradas para el cine. detrás de esas cosas casi siempre hay un programa corriendo que organiza la información para que todo sea más rápido y con menos errores.

Para el proyecto final en la Materia Lógica de Programación se nos pidió justamente eso: pensar en un problema real de la vida diaria y resolverlo, aplicando todo lo que se trabajó en las 4 unidades del semestre. El profesor puso como ejemplo un cajero automático y una lavadora, busque un problema distinto donde se pudiera aplicar la misma lógica, y termine decidiéndome por un sistema de reservas de asientos para una sala de cine.

En este documento se explicará cómo se planteó el problema, qué relación tiene con lo aprendido en clase, cómo se desarrolló el sistema y qué aprendizajes obtuvimos, especialmente al reflexionar sobre el impacto de la tecnología en una actividad cotidiana como ir al cine.

2. OBJETIVOS

2.1 Objetivo General

Desarrollar un sistema de reservas de asientos para una sala de cine mediante un programa en Python, que permita gestionar de manera sencilla la disponibilidad de asientos desde la consola, aplicando los conocimientos adquiridos en la materia de Lógica de Programación y relacionándolos con un problema de la vida cotidiana.

2.2 Objetivos Específicos

- Analizar un problema real relacionado con la reserva de asientos en una sala de cine.
- Aplicar los conceptos aprendidos en las diferentes unidades de la materia.
- Reflexionar sobre el impacto de la tecnología en la vida diaria.



3. METODOLOGÍA

La metodología utilizada en el presente proyecto consistió en la identificación y análisis de un problema de la vida cotidiana, seguido de la aplicación de los conocimientos adquiridos en la materia de Lógica de Programación.

Posteriormente, se procedió el diagrama y el código del sistema, realizando pruebas y ajustes continuos con el fin de garantizar su correcto funcionamiento.

4. RESULTADOS

4.1 Description del problema

Cualquiera que haya ido al cine sabe cómo es cuando no hay un sistema bien organizado. Uno llega, pregunta qué asientos están libres y la persona que atiende tiene que ponerse a revisar, a veces en un papel o incluso de memoria. Todo se vuelve lento y un poco confuso. Y si dos personas piden el mismo asiento casi al mismo tiempo, ahí es donde empiezan los problemas, porque no siempre está claro quién lo pidió primero.

Con base en esta idea, se diseñó un sistema que busca resolver este problema de manera automatizada, evitando que una persona tenga que gestionarlo manualmente. El sistema permite controlar qué asientos están ocupados y cuáles disponibles, evitando la duplicación de reservas y brindando al encargado de la sala una visualización clara y rápida de la información.

5 Relacion con los contenidos de la asignatura

Unidad 1 - Fundamentos de programación

Para empezar, todo el programa se apoya en variables: el código de cada asiento, el nombre del cliente, el total de asientos, entre otras. también se usan operadores, por ejemplo para calcular el porcentaje de ocupación de la sala, que básicamente es una regla de tres hecha con código.

Unidad 2 - Estructuras de control

El programa necesita repetirse hasta que el usuario decida salir, así que el menú principal esta armado con un bucle while. Junto con eso, casi todas las funciones usan condicionales (if, elif, else) para revisar cosas como si el asiento existe o si ya estaba ocupado antes de hacer cualquier cambio.



Unidad 3 - Estructuras de datos

Aquí esta, podría decirse, el corazón del programa: toda la sala de cine se representa con un diccionario, donde cada asiento (por ejemplo "A1") tiene guardado si esta ocupado y quien lo reservo. Se eligió un diccionario y no una lista porque así se puede ir directo a un asiento por su código, sin tener que recorrer todos los demás para encontrarlo. también se usan bucles for para recorrer ese diccionario cuando se muestra el mapa de asientos o se calcula la ocupación.

Unidad 4 - Funciones y manejo de errores

Por último, todo el programa está dividido en funciones, una por cada tarea (reservar_asiento, cancelar_reserva, consultar ocupación, buscar reserva, entre otras), lo que ayuda a que el código no sea un solo bloque gigante y se entienda mejor. Además, se uso try/except en la

6. Explicación del sistema desarrollado

El sistema, al que llamamos CineReserva, corre completamente desde la consola, sin necesidad de una interfaz gráfica, tal como indico el profesor para este nivel del curso. Cuando se ejecuta el programa, se crea automáticamente una sala con 20 asientos (filas A,D, 5 asientos por fila) y se muestra un menú con 6 opciones.

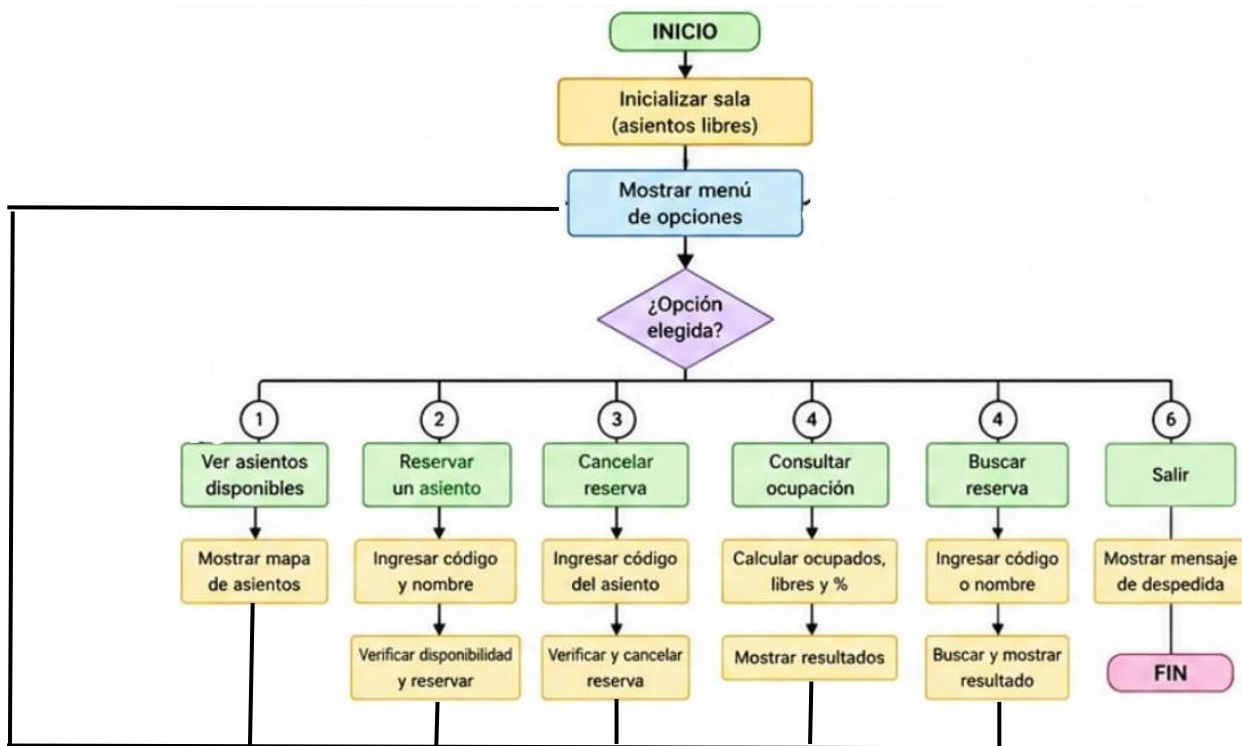
Antes de escribir el código, en como representar la sala. La primera idea fue usar una lista simple con los números de los asientos, pero al pensarlo mejor me di cuenta de que buscar un asiento específico seria más lento y enredado, así que termine optando por el diccionario que se explicó en la sección anterior.

#	Funcionalidad	Descripción
1	Ver asientos disponibles	Muestra el mapa completo de la sala (20 asientos, filas A-D) indicando cuales están libres y cuales ocupados.
2	Reservar un asiento	El usuario ingresa el código del asiento (ej. A1) y su nombre; el sistema valida que el asiento exista y este libre.
3	Cancelar una reserva	Libera un asiento que estaba reservado.
4	Consultar ocupación	Muestra el total de asientos ocupados, libres y el porcentaje de ocupación de la sala.
5	Buscar una reserva	Busca por código de asiento o por nombre del cliente.
6	Salir	Termina la ejecución del programa.



7 Diagrama de funcionalidad

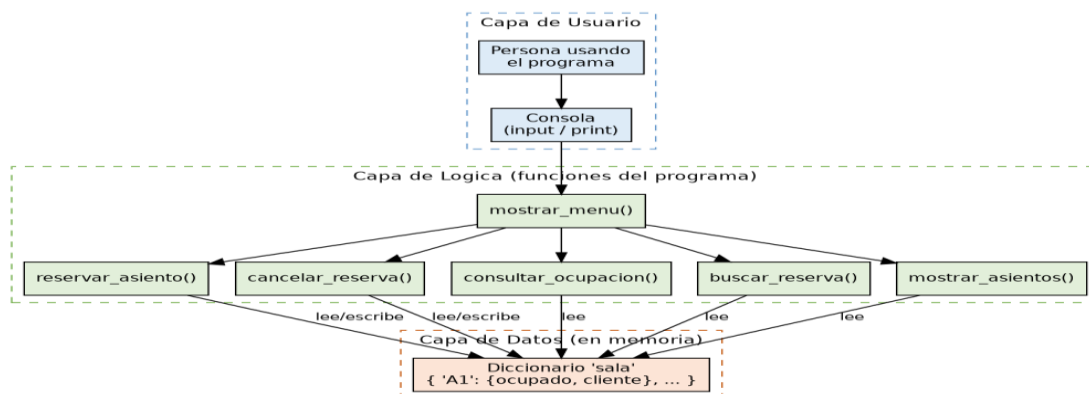
Antes de programar se hizo este diagrama para organizar el flujo completo del programa, desde que inicia hasta que el usuario decide salir. Después de cada opción el programa siempre regresa al menú, excepto cuando se elige la opción 6 (Salir).



8 Diagrama de arquitectura

Este diagrama muestra como quedo organizado el sistema en 3 capas: la capa de usuario (donde la persona interactúa con la consola), la capa de lógica (las funciones que hacen el trabajo) y la capa de datos (el diccionario que guarda todo en memoria mientras el programa está corriendo). No se usó base de datos real ni una arquitectura cliente-servidor porque, como explico el profesor, ese tipo de arquitectura en capas se ve más adelante, en semestres más avanzados.

Diagrama de Arquitectura - Sistema de Reservas de Cine





9 Sobre el código y el control de versiones

El código se organizó en funciones independientes, cada una comentada explicando que recibe (input) y que devuelve o muestra (output), tal como se pidió en clase. Todo el proyecto, es decir el código, los diagramas, el README y este mismo documento, se subió a un repositorio de GitHub para llevar un control de versiones ordenado, tal como se indicó en las instrucciones del proyecto.

10 Reflexión sobre el impacto de la tecnología

Haciendo este proyecto nos quedó más claro algo que el profesor mismo menciona al presentar el trabajo: las nuevas tecnologías están cambiando la forma en que la gente resuelve problemas del día a día, aunque la mayoría de las veces esos cambios pasan casi sin que los notemos. Comprar una entrada de cine, lo que antes era simplemente hacer fila y esperar, hoy se puede resolver desde una aplicación en el celular, y detrás de esa aplicación hay, en el fondo, la misma lógica que se aplicó en este proyecto: validar datos, guardar información y organizar todo con código.

Me di cuenta de que para automatizar algo no se necesita un programa gigante ni complicado. Con condicionales, bucles, una estructura de datos bien pensada y funciones organizadas, ya se puede resolver un problema real, así sea a una escala pequeña como la de este proyecto.

Este trabajo deja una idea bastante clara: antes de ponerse a programar hay que entender bien el problema y pensar cómo se va a organizar la solución, como se hizo aquí con los diagramas, y solo después de eso programar de forma ordenada y bien comentada, para que cualquier otra persona pueda entender el código sin tener que preguntar.

Conclusiones

En conclusión, el desarrollo del sistema *CineReserva* permitió aplicar de manera práctica los conocimientos adquiridos en la asignatura de Lógica de Programación, demostrando que es posible resolver problemas cotidianos mediante soluciones tecnológicas simples pero eficientes. A través de este proyecto, se logró diseñar e implementar un sistema capaz de gestionar la reserva de asientos de manera organizada, evitando errores como la duplicación de reservas y mejorando la experiencia del usuario.

Además, el uso de estructuras de datos como diccionarios, junto con el diseño previo mediante diagramas, facilitó la comprensión y construcción del sistema. El trabajo también permitió fortalecer habilidades importantes como la lógica, el análisis de problemas y la organización del código.



Finalmente, este proyecto evidencia la importancia de la programación como herramienta para optimizar procesos en la vida real, dejando como base la posibilidad de futuras mejoras, como la implementación de interfaces gráficas o conexión con bases de datos en niveles más avanzados.

BIBLIOGRAFÍA

- Python Software Foundation. (2026). Documentación oficial de Python. <https://www.python.org/doc/>
- GitHub. (2026). Documentación oficial de GitHub. <https://docs.github.com/>
- Material de clase de la asignatura Lógica de Programación. Docente: Darío Sebastián Cabezas Erazo.
- Fundamentos de programación: conceptos de variables, estructuras de control y funciones.
- Estructuras de datos: uso de diccionarios y listas en programación.